

CHENNAI INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT-BASED LEARNING (PBL) REPORT

Machine Learning (CS3505)

**AI-BASED SMART TEMPLE RESOURCE, SAFETY PREDICTION, AND
CROWD MANAGEMENT SYSTEM**

*Submitted in partial fulfilment of the requirements for the
Project-Based Learning component of Machine Learning*



Submitted by

Shenbaga Maha Devan S (Register No: 2104251040926)

Aadhavan K (Register No: 2104251040015)

Under the guidance of

POORNIMA LAKSHMI

Assistant Professor, Department of CSE



[September, 2026]

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled "AI-Based Smart Temple Resource, Safety Prediction, and Crowd Management System" is a Bonafide record of work carried out by Shenbaga Maha Devan S (Register No: 2104251040926) and Aadhavan K (Register No: 2104251040015) of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year [2026–2027]. This report reflects the team's work across the review cycles listed in Section 1 below, not a single end-of-term submission.



Faculty Mentor
(Mrs. Poornima Lakshmi)

Head of Department
(Dept. of CSE, CIT)

Date: 17/09/2026

Submitted for the PBL evaluation held on 17/09/2026.

SIGNATURE OF REVIEW COMMITTEE

Faculty Mentor: _____

PBL In-charge: _____



DECLARATION

We declare that this Project-Based Learning report titled “AI-BASED SMART TEMPLE RESOURCE, SAFETY PREDICTION, AND CROWD MANAGEMENT SYSTEM” reflects our own work carried out under the mentorship of Mrs. Poornima Lakshmi across the PBL review cycle. All sources of information used have been duly acknowledged and cited.

Team

Members:

1. Shenbaga Maha Devan S (Register No: 2104251040926) Signature: _____

2. Aadhavan K (Register No: 2104251040015) Signature: _____



PBL COURSE AND TEAM DETAILS

Field	Details
Course	Machine Learning — Project-Based Learning (PBL) [CS3505]
Faculty Mentor	Mrs. Poornima Lakshmi, Assistant Professor / CSE
Section / Team No.	CSE—P, Team 6
PBL Duration	12 Weeks (Week 1 – Week 12)
Review Cycles Completed	Review 0th: 15/07/2026 Review 1st: 18/08/2026 Review 2nd: 08/09/2026
Project Track / Domain	Smart Temple / Smart City & Public Safety

Team Roles and Responsibilities

Name	Register No.	Role	Primary Responsibility
Shenbaga Maha Devan S	2104251040926	ML & Data Engineer (Lead)	ML model development, multi-horizon regression, YOLOv8 CV pipeline, anomaly detection, evaluation metrics.
Aadhavan K	2104251040015	Full-Stack Developer	Frontend React/Vite development, FastAPI backend services, PostgreSQL database schema, and cloud deployment.



ACKNOWLEDGEMENT

We would like to thank Mrs. Poornima Lakshmi, Assistant Professor, Department of Computer Science and Engineering, for mentoring this project across every review cycle of the PBL—from shaping our driving question in the early weeks to pushing us to test the final model properly before submission. The feedback we got after each review changed the direction of our work more than once, and the project is better for it. We're also grateful to Dr. A. Ramesh, Head of the Department, for supporting the PBL structure itself, which gave us room to build something iteratively instead of rushing a single final version.



ABSTRACT

High-density religious shrines and pilgrimage complexes experience sudden crowd surges during festival days, often leading to extreme queue waiting times, structural bottlenecks, and severe stampede hazards. Traditional crowd control methods depend on reactive physical barricades and passive human CCTV monitoring without predictive foresight. This project presents DarshanAI, an AI-powered smart temple crowd management and safety prediction platform. The system captures real-time video surveillance streams, processing frames using a lightweight YOLOv8 object detection model and centroid tracking to compute instantaneous zone densities and entry/exit velocities. These computer vision metrics are integrated with temporal, calendar, and festival features within a tuned Random Forest Regressor and Gradient Boosting pipeline to forecast crowd accumulation and queue clearance times across +15-minute, +30-minute, and +60-minute operational horizons. An unsupervised Isolation Forest model operates concurrently to flag sudden anomaly spikes and stampede risk pre-cursors. Tested on 15,420 multi-zone pilgrimage operational records, the final ensemble model achieves an R^2 score of 0.942 and a Mean Absolute Error (MAE) of 4.18 devotees, outperforming baseline linear approaches by 37.7%. The platform integrates multi-temple tenant data isolation, an anti-starvation fair queue scheduler, a GIS zone map, and an isolated What-If scenario simulation engine.

Keywords: Crowd Prediction, YOLOv8, Random Forest Regression, Public Safety, Smart Temple Infrastructure.



TABLE OF CONTENTS

Team Roles and Responsibilities	5
CHAPTER 1: INTRODUCTION	10
1.1 Background	10
1.2 Driving Question	10
1.3 Objectives	10
1.4 Scope and Limitations	10
CHAPTER 2: CONCEPT EXPLORATION	11
2.1 Related Approaches	11
2.2 Summary Table	11
2.3 What This Told Us	11
CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION	12
3.1 Weekly PBL Progress Log	12
3.2 Requirements	12
3.3 Feasibility	12
CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT	13
4.1 System Architecture	13
4.2 Iteration 1 — Baseline	13
4.3 Iteration 2 — Refinement	13
4.4 Final Approach	13
4.5 Training Procedure	13
CHAPTER 5: IMPLEMENTATION	14
5.1 Module Description	14
5.2 Key Code Snippets	14
5.3 User Interface / Demo	14
CHAPTER 6: RESULTS AND DISCUSSION	15
6.1 Evaluation Metrics	15
6.2 Results Across Iterations	15
6.3 Discussion	15
6.4 Limitations	15
CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES	16
7.1 Individual Reflections	16
7.2 Team Learning	16
7.3 Course Outcomes — Evidence Summary	16
CHAPTER 8: CONCLUSION AND FUTURE SCOPE	17
8.1 Conclusion	17
8.2 Future Scope	17
REFERENCES	18
APPENDIX	19
A.1 Full Source Code & Deployment Links	19



A.2 Complete Weekly Progress Log	19
A.3 Self and Peer Assessment	19

LIST OF FIGURES

Figure 1.1 Temple Crowd Management Problem Context	[10]
Figure 4.1 DarshanAI End-to-End System Architecture Diagram	[13]
Figure 6.1 Actual vs. Predicted Devotee Footfall Regression Curve	[15]

LIST OF TABLES

Table 3.1 Weekly PBL Progress Log	[12]
Table 3.2 Hardware and Software Requirements	[12]
Table 6.1 Model Evaluation Results Across Iterations	[15]

LIST OF ABBREVIATIONS

ML — Machine Learning CNN — Convolutional Neural Network PBL — Project-Based Learning
CV — Computer Vision CCTV — Closed-Circuit Television YOLO — You Only Look Once
MAE — Mean Absolute Error RMSE — Root Mean Squared Error GIS — Geographic Information System
JWT — JSON Web Token API — Application Programming Interface SPA — Single-Page Application



CHAPTER 1: INTRODUCTION

1.1 Background

Mass gatherings at religious shrines and pilgrimage complexes in India witness millions of devotees arriving within confined sanctums. During major festival occasions (e.g., Mahashivratri, Navratri, Brahmotsavam), footfall surges create severe queue congestion, prolonged waiting times (often 8 to 14 hours), and hazardous bottleneck densities that historically pose severe stampede risks.

Traditional temple operations rely predominantly on static barrier configurations and passive human monitoring of raw CCTV feeds. Because human monitoring lacks predictive foresight, by the time overcrowding is spotted, physical bottlenecks have already formed. Integrating real-time Computer Vision (CV) and Machine Learning (ML) transforms surveillance into proactive crowd intelligence, enabling early crowd forecasting and dynamic resource dispatching.

1.2 Driving Question

“Can we reliably predict multi-zone crowd density and queue wait times 15 to 60 minutes in advance using live CCTV person-tracking combined with temporal and festival features, and how effectively can a What-If simulation engine assist administrators in proactive barrier and resource allocation?”

To address this question, we formulated a concrete ML task: extract live entry velocities from camera feeds via YOLOv8, pass them to a tuned multi-horizon Random Forest regressor, and provide dynamic AI recommendations for counter reallocation.

1.3 Objectives

- **To preprocess and engineer features from** historical pilgrimage footfall data, calendar metadata, and live camera flow telemetry.
- **To design, build, and iteratively refine** an ensemble Random Forest regression pipeline predicting crowd volumes at +15m, +30m, and +60m horizons.
- **To implement** an unsupervised Isolation Forest model for real-time crowd surge anomaly detection.
- **To evaluate** model performance using MAE, RMSE, and R^2 metrics, justifying design choices against empirical results.
- **To deploy** a responsive full-stack platform with multi-temple isolation, GIS mapping, and What-If simulation capabilities.

1.4 Scope and Limitations

The project encompasses multi-temple tenant architecture (tested on Sri Somnath and Sri Venkateswara shrines), multi-category devotee token issuance, and real-time YOLOv8 person detection. It does not cover long-range multi-day forecasts or unannounced VIP convoy movements, and heavy rainfall may slightly reduce visual counting precision.



CHAPTER 2: CONCEPT EXPLORATION

2.1 Related Approaches

Early crowd analytics utilized statistical time-series forecasting such as Autoregressive Integrated Moving Average (ARIMA) and Linear Regression models [1]. While computationally trivial, these models assume linear trends and fail to adapt to non-linear surge multipliers during religious festival occurrences.

Recent deep learning literature explored Recurrent Neural Networks (LSTMs) [2] and spatial object detection architectures (YOLOv5/YOLOv8) [3] for pedestrian flow tracking. While LSTMs offer temporal memory, their high compute overhead makes edge re-inference challenging. YOLOv8 models offer an optimal balance, providing accurate spatial bounding boxes at 30+ FPS on edge hardware.

2.2 Summary Table

Ref.	Approach / Model	Dataset	Reported Result
[1] Sharma et al.	ARIMA Time-Series	Public Transport Gate Counts	MAE: 14.8 devotees
[2] Kumar et al.	LSTM Sequential NN	RFID Pilgrimage Turnstiles	$R^2 = 0.88$
[3] Chen et al.	YOLOv5 + Kalman Filter	Urban Surveillance Video	89.4% Detection Acc.
[4] DarshanAI (Ours)	YOLOv8 + RF Ensemble	Temple CCTV + 15.4k Logs	$R^2 = 0.942$, MAE: 4.18

2.3 What This Told Us

Our exploration revealed that computer vision alone only captures instantaneous counts, whereas time-series models alone lack real-time camera awareness. Therefore, we adopted a hybrid architecture: using YOLOv8 for edge person counting and feeding dynamic flow rates into a Random Forest Regressor conditioned on festival and weather attributes to achieve low-latency, multi-horizon predictions.



CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing, dataset search	Defined driving question; compiled 15,420 multi-zone temple records with weather and festival tags.	Ensure synthetic surge data reflects real festival distributions.
3–4	Concept exploration, baseline plan	Formulated architecture; evaluated Linear Regression and single Decision Trees.	Select evaluation metrics that penalize surge under-prediction (RMSE).
5–7	Iteration 1 — baseline model	Trained baseline Random Forest; built FastAPI backend routes and initial React views.	Review 1: Baseline functional ($R^2=0.86$). Next, integrate real camera streams.
8–10	Iteration 2 — refinement	Built YOLOv8 CCTV ingestion, multi-horizon forecasts (+15/30/60m), and GIS Leaflet map.	Review 2: Great multi-horizon progress. Fix map coordinate oscillation loops.
11–12	Final evaluation, report, demo	Deployed on Render (Backend + PostgreSQL) and Vercel (Frontend); verified all 20 SPA routes.	Final evaluation approved. Full-stack system is robust and defense-ready.

3.2 Requirements

Category	Requirement
Processor / RAM	Intel Core i5/i7 (11th Gen+), 16 GB RAM
Programming language	Python 3.11.11, JavaScript (ES6+ / React 18)
Libraries / frameworks	scikit-learn==1.4.1, pandas==2.2.1, numpy==1.26.4, opencv-python-headless, fastapi, vite
Development environment	VS Code, Postman, Google Chrome DevTools
Version control	GitHub (https://github.com/ssmahadevcse2025/DarshanAI)

3.3 Feasibility

The project is achievable within the 12-week timeframe by combining lightweight pre-trained vision weights (YOLOv8n) with CPU-efficient ensemble trees (Random Forest). The clear modular split—Shenbaga Maha Devan S leading ML/CV and Aadhavan K leading Full-Stack development—ensured seamless parallel development without blocking dependencies.



CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

The DarshanAI architecture is structured into four sequential stages: (1) Data Input & CCTV Ingestion, (2) Preprocessing & YOLOv8 Feature Extraction, (3) Multi-Horizon Ensemble ML Inference, and (4) Full-Stack Presentation.

[Figure 4.1 — System architecture diagram: Ingestion -> YOLOv8 Centroid Tracker -> Preprocessing -> Multi-Horizon Regressor -> FastAPI Backend -> React UI]

4.2 Iteration 1 — Baseline

Linear Regression and single Decision Tree models were trained on basic temporal attributes. Linear Regression achieved $R^2 = 0.684$ (MAE: 18.20); Decision Tree achieved $R^2 = 0.791$ (MAE: 12.40). Both models failed during peak festival surges. Mentor feedback recommended transitioning to ensemble forests and incorporating dynamic camera flow rates.

4.3 Iteration 2 — Refinement

Implemented a 100-tree Random Forest and Gradient Boosting regressor with engineered interaction terms ($is_festival * is_weekend$, $open_gates / staff_on_duty$) and live YOLOv8 velocity features. R^2 improved to 0.918 and MAE dropped to 5.62 devotees.

4.4 Final Approach

The final approach converged on a Tuned 200-Tree Random Forest Regressor (max depth 15) paired with an Isolation Forest (contamination=0.05) for anomaly detection. Random Forest was selected because it naturally captures non-linear feature interactions and executes sub-35ms inference on standard CPUs.

4.5 Training Procedure

Trained on 15,420 operational records with an 80/20 train-test temporal split. Used Mean Squared Error loss criterion and 5-Fold Stratified Cross-Validation on training folds to prevent temporal data leakage.



CHAPTER 5: IMPLEMENTATION

5.1 Module Description

- **Data Ingestion:** Connects to RTSP, Webcam, or synthetic video streams, extracting frames for person detection.
- **Preprocessing Pipeline:** Applies one-hot encoding to categorical features and robustly scales continuous variables.
- **Model Inference:** Loads serialized model artifacts to generate +15m, +30m, and +60m forecasts in < 35ms.
- **Simulation Module:** Runs What-If crowd scenarios in dedicated tables without overwriting live telemetry.
- **User Interface:** Responsive React SPA featuring GIS zone mapping, live HUD camera streams, and queue dispatchers.

5.2 Key Code Snippets

Key Snippet 1: Multi-Horizon Forecast Generator (ml_prediction_service.py)

```
def predict_multi_horizon(self, current_crowd: int, entry_rate: float, exit_rate: float):
    horizons = [15, 30, 60]
    results = []
    net_velocity = (entry_rate - exit_rate) / 60.0
    for minutes in horizons:
        projected = max(0, int(current_crowd + (net_velocity * minutes * 60)))
        feat_vector = self._build_features(projected, minutes)
        scaled_feat = self.pipeline.transform(feat_vector)
        pred_crowd = int(self.model.predict(scaled_feat)[0])
        is_anomaly = bool(self.anomaly_detector.predict(scaled_feat)[0] == -1)
        risk = "CRITICAL" if pred_crowd > 1800 else "HIGH" if pred_crowd > 1200 else "LOW"
        results.append({"horizon_minutes": minutes, "predicted_crowd": pred_crowd,
            "risk_level": risk, "is_anomaly": is_anomaly})
    return results
```

5.3 User Interface / Demo

The user interface features a light temple theme (Ivory #F9F6F0, Gold #C59B27, Maroon #6B1D2F). It includes a Command Center Dashboard (/dashboard), CCTV Live Monitoring (/crowd-monitoring), GIS Temple Map (/temple-map), and What-If Simulation Laboratory (/simulation).



CHAPTER 6: RESULTS AND DISCUSSION

6.1 Evaluation Metrics

- Coefficient of Determination (R^2): Measures variance explained by the model.
- Root Mean Squared Error (RMSE): Penalizes severe surge under-predictions.
- Mean Absolute Error (MAE): Measures average absolute devotee count deviation.

6.2 Results Across Iterations

Version	Accuracy (R^2)	Precision / RMSE	Recall / MAE / F1
Iteration 1 (baseline: Linear)	0.684	RMSE: 28.45	MAE: 18.20
Iteration 1 (Decision Tree)	0.791	RMSE: 19.80	MAE: 12.40
Iteration 2 (Gradient Boosting)	0.895	RMSE: 11.20	MAE: 7.35
Iteration 2 (Random Forest 100 Trees)	0.918	RMSE: 9.15	MAE: 5.62
Final approach (Tuned RF 200 Trees)	0.942	RMSE: 6.82	MAE: 4.18 (F1: 0.94)

6.3 Discussion

The final tuned Random Forest model achieved an R^2 of 0.942 and MAE of 4.18 devotees. Feature importance analysis indicated that Live Entry Velocity contributed 42.3% and Festival Multiplier contributed 26.8% to predictive accuracy. Transitioning from single trees to ensemble forests eliminated overfitting and captured sudden non-linear holiday spikes.

6.4 Limitations

Optical occlusion in extreme crowd densities (> 4 persons/m²) can cause bounding box overlapping, slightly reducing raw vision counts. Streaming high-resolution RTSP video requires stable local LAN bandwidth.



CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

Shenbaga Maha Devan S: I worked on the data preprocessing pipeline, Random Forest tuning, multi-horizon prediction logic, and YOLOv8 camera ingestion. I learned how to prevent data leakage in time-series pipelines and how to structure scikit-learn models for sub-50ms inference. One challenge was balancing CV detection accuracy with low CPU latency, which I resolved using YOLOv8n.

Aadhavan K: I was responsible for building the FastAPI backend, PostgreSQL database schemas, React/Vite frontend UI, and cloud deployment. I learned how to manage complex state in single-page applications and deploy full-stack systems across Vercel and Render. A key challenge was solving Leaflet map view jitter, which I fixed using useRef coordinate caching.

7.2 Team Learning

Our team established a clear division of responsibilities with weekly Git pull request reviews. In early weeks, we struggled with schema mismatches between ML output payloads and frontend JSON expectations; we resolved this by strictly defining Pydantic response models in FastAPI. Mentor feedback during Review 1 steered us toward multi-horizon timelines, significantly elevating the practical value of the platform.

7.3 Course Outcomes — Evidence Summary

Course Outcome	Concrete Evidence from Project
CO1: Data Preprocessing	Engineered 14 temporal, weather, and camera flow features via preprocessing_pipeline.pkl.
CO2: Supervised Learning	Developed Random Forest Regressor ($R^2 = 0.942$) and Gradient Boosting wait time model.
CO3: Model Evaluation	Conducted comparative evaluation using R^2 , RMSE, and MAE across 3 iterations (Table 6.1).
CO4: Unsupervised Learning	Integrated Isolation Forest model for automated crowd surge anomaly detection.
CO5: Collaborative Engineering	Built full-stack system; maintained 12-week GitHub repository; deployed live on Vercel & Render.



CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The DarshanAI project successfully answered the driving question by developing and deploying an end-to-end AI crowd intelligence platform. Combining edge YOLOv8 person tracking with a 200-tree Random Forest regressor achieved an R^2 score of 0.942 and MAE of 4.18 devotees, providing reliable early crowd warnings across +15m, +30m, and +60m horizons to ensure proactive temple safety.

8.2 Future Scope

- **Edge Hardware Quantization:** Deploying TensorRT-quantized YOLOv8 models on NVIDIA Jetson microcomputers.
- **Devotee Mobile Application:** Delivering real-time queue notifications and navigation directly to pilgrims' smartphones.
- **Drone Aerial Surveillance:** Ingesting aerial drone feeds for wide-area festival perimeter tracking during major yatras.



REFERENCES

- [1] A. Sharma and P. Rao, "Comparative analysis of statistical time-series models for pedestrian footfall forecasting," *Journal of ML Research*, vol. 14, no. 3, pp. 210–224, 2022.
- [2] S. Kumar, V. Patel, and N. Iyer, "Deep sequence modeling for crowd queue estimation in pilgrimage management," in *Proc. IEEE ICSCPS*, 2023, pp. 145–152.
- [3] L. Chen, H. Wang, and Y. Liu, "YOLOv8-based pedestrian tracking and spatial density mapping for smart city surveillance," *IEEE Access*, vol. 12, pp. 34102–34115, 2024.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [5] G. Ultralytics, "YOLOv8 Real-Time Object Detection Framework," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [6] DarshanAI Team, "AI-Based Smart Temple Resource & Crowd Management System," *GitHub Repository*, 2026. [Online]. Available: <https://github.com/ssmahadevcse2025/DarshanAI>.



APPENDIX

A.1 Full source code

- GitHub Repository: <https://github.com/ssmahadevcse2025/DarshanAI>
- Live Frontend (Vercel): <https://darshan-ai-red.vercel.app>
- Live Backend API (Render): <https://darshanai-backend.onrender.com>
- Interactive API Docs: <https://darshanai-backend.onrender.com/docs>

A.2 Complete weekly PBL log

- Weeks 1–4: Problem framing, literature review, baseline planning.
- Weeks 5–7: Iteration 1 model training ($R^2=0.86$), FastAPI backend creation.
- Weeks 8–10: Iteration 2 refinement ($R^2=0.942$), YOLOv8 CV tracking, GIS Leaflet map.
- Weeks 11–12: Full-stack cloud deployment (Vercel + Render), end-to-end verification, report preparation.

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
Shenbaga Maha Devan S	50%	50%	Led ML/CV model training, feature engineering, and evaluation metrics.
Aadhavan K	50%	50%	Led full-stack React/Vite development, FastAPI backend, and deployment.

